Projet ben

📋 Dossier de Conception : Application de Gestion de Plannings (Trame RH)

1. Vision du Projet

- **Objectif :** Digitaliser et automatiser les roulements récurrents (cycles de 12h, 2×8, 3×8).
- **Problème :** Complexité de calcul de l'**effectif instantané** et erreurs de saisie sur les cycles longs (type 14 jours).
- **Cible :** Cadres de santé, managers de logistique, secteur industriel.

---

2. Architecture des Données (Base de Données)

Dans Notion, crée ces trois bases de données liées :

🗄️ Table : Employés

Propriété	Type	Description
Nom	Titre	Nom de l'agent
Trame Associée	Relation	Lien vers la table "Trames"
Date de début	Date	Quand l'agent commence son cycle
Contrat	Sélection	100%, 80%, 50%

Exporter vers Sheets

🗄️ Table : Trames (Le Modèle Théorique)

C'est ici que tu définis le "modèle" de référence.

Propriété	Type	Exemple
Nom de la Trame	Titre	Roulement 12h - Petite/Grande Semaine
Durée du Cycle	Nombre	14 (jours)
Description	Texte	Alternance 2-2-3 (Mardi/Mercredi... etc)

---

3. Modélisation du Roulement

*Copie ce bloc dans un bloc "Code" sur Notion et sélectionne le langage **Mermaid**.*

Extrait de code

```
graph TD
    subgraph "Trame Théorique (14 jours)"
    A[Semaine 1 : Petite] -->|Poursuite| B[Semaine 2 : Grande]
    B -->|Récurrence| A
    end

    subgraph "Calcul d'Effectif"
    A --> C{Compteur Présence}
    B --> C
    C --> D[Effectif Instantané / Jour]
    end
```

```
    subgraph "Planning Réel"
    D --> E[Ajustement Manuel : Congés/Maladie]
    end
```

---

4. Détail du Cycle "Jo_bis" (Exemple 12h)

Voici la trame type à insérer comme exemple de référence dans ton document :

Semaine	Lundi	Mardi	Mercredi	Jeudi	Vendredi	Samedi	Dimanche
S1 (Petite)	12h	Repos	Repos	12h	12h	Repos	Repos
S2 (Grande)	Repos	12h	12h	Repos	Repos	12h	12h

Exporter vers Sheets

> **Note technique :** Le calcul de l'**Effectif Présent** à un instant T se fait par la formule :
>
> Effectif=∑(Agents dont le jour du cycle=Travail)

---

5. Fonctionnalités Clés (User Stories)

1. **Génération Auto :** "En tant que manager, je veux que le planning se remplisse tout seul sur 6 mois à partir de la trame choisie."
2. **Alerte Repos :** "Le système doit bloquer si un agent travaille plus de 48h sur une semaine isolée."
3. **Visualisation :** Vue "Calendrier" pour l'employé et vue "Tableau de bord" (Effectif par jour) pour le manager.

💡 FICHIER BEN

À partir du fichier `planing 12.ods` que tu as partagé et de nos échanges sur le roulement en 12h, voici une synthèse complète pour ton espace **Notion**.

Le fichier contient les briques de base d'une gestion de temps de travail en 12h.

---

1. Modélisation de la Trame (Structure de données)

Dans Notion, pour répliquer l'intelligence de ton fichier `.ods`, tu dois structurer tes données ainsi :

Objets Principaux

- **L'Entité "Cycle" :** Un bloc de 14 jours (2 semaines).
- **L'Entité "Shift" :** Définit la vacation (ex: "Jour 12h", "Nuit 12h", "Repos").
- **L'Entité "Affectation" :** Le lien entre un employé, une trame et une date de début (pivot).

Logique du Roulement "2-2-3" (Inspiré du fichier)

Voici comment le cycle se décompose mathématiquement sur 14 jours :

- **Semaine A :** Travail (Lun, Jeu, Ven) | Repos (Mar, Mer, Sam, Dim).

- **Semaine B :** Travail (Mar, Mer, Sam, Dim) | Repos (Lun, Jeu, Ven).

---

2. Use Case : Calcul de l'Effectif Cible (Le "Need")

Titre : Monitoring de la couverture de soins/service.
Acteur : Responsable d'unité / Manager.

Scénario :

1. Le manager ouvre sa page Notion "Tableau de Bord Quotidien".
2. Le système calcule automatiquement quel employé est en "Semaine A" et lequel est en "Semaine B" par rapport à la date du jour.
3. **Résultat attendu :** L'application affiche le nombre de RH présents (Effectif instantané).

---

3. Ce que l'on peut tirer du fichier (Analyse technique)

Le fichier `.ods` révèle plusieurs couches de données essentielles:

+1

- **Gestion des compteurs :** Le format tableur est conçu pour sommer les heures. Dans Notion, il faudra utiliser une `Rollup` pour calculer le total d'heures mensuelles par agent afin de vérifier s'ils dépassent les plafonds légaux.
- **Visualisation Matricielle :** Le fichier utilise une grille. Dans Notion, cela correspondra à une vue **Timeline** (Chronologie) groupée par équipe.

  +1
- **Périodicité :** Le contenu suggère une répétition infinie. Pour ton app, cela signifie qu'il ne faut pas créer de lignes manuellement, mais utiliser une formule qui projette la trame sur le calendrier.

---

4. Modélisation Mermaid pour Notion

Copie ce code dans un bloc Notion pour visualiser la logique de génération du planning :

Extrait de code

```
graph LR
    A[Date du Jour] --> B{Formule Modulo 14}
    B -->|Résultat 1-7| C[Semaine Petite]
    B -->|Résultat 8-14| D[Semaine Grande]

    C --> E[Check Trame Théorique]
    D --> E

    E --> F{Est-ce un jour ON?}
    F -->|Oui| G[Afficher en Poste]
    F -->|Non| H[Afficher en Repos]
```

---

5. Ce que je peux faire de plus pour toi

À partir des éléments du fichier `planing 12.ods`, je peux te générer :

1. **La Formule Notion (JSON/Texte) :** Pour calculer automatiquement le statut (Travail/Repos) selon la date.
2. **Un Guide de saisie :** Pour que tes utilisateurs ne cassent pas la trame récurrente.

Souhaites-tu que je te rédige la formule spécifique pour Notion afin qu'elle gère exactement le cycle de 12h de ton fichier ?

---

1. Vision et Objectifs

- Nom du projet : USM unit schedule Manager
- Problème à résoudre : Quel est le "pain point" principal ?
  - generer les planning d'un service de soin en prenant en compte le plus de critere possible
  - générer
- Public cible : cadre et agents
- Proposition de valeur : Pourquoi utiliser ton app plutôt qu'une autre ?

2. Analyse Fonctionnelle

- Fonctionnalités principales (MVP) : Les fonctions indispensables pour le lancement.
- Fonctionnalités secondaires : Ce qui sera ajouté dans les versions futures.
- Règles de gestion : (Exemple : "Un utilisateur doit avoir 18 ans pour s'inscrire", "Le panier expire après 24h").

3. Architecture de l'Information

- Sitemap : Arborescence des pages/écrans.
- User Flow : Schéma du parcours type d'un utilisateur (de l'inscription à l'action finale).

4. Modélisation des Données (Backend)

C'est ici que tu définis comment les informations sont stockées.

- Dictionnaire des données : Liste des objets
  - planning :
    - roulement réel de tous les agent du service en tenant compt de l'effectif
  - trame :
    - roulement récurrent théorique du travail sur un période donnée

Attribut	Description	Exemple
Nom du service	Identifiant du roulement.	"Équipe 2×8 A"
Périodicité	Durée du cycle en jours ou semaines.	12 semaines
Trame	Liste ordonnée des shifts.	[L:Matin, M:Matin... D:Repos]
Date de pivot	Date de départ pour caler le cycle dans le temps.	01/01/2024
Effectifs	nombre d'agent présent à chaque instant en fonction des jours et des horaires	lundi: 2 agent de jour, 2 de nuit, mardi

- **service de soin: lieux ou les agents travail**
- **infirmière : agent qui travail en fonction du planning**
- **cadre de santé : agent qui génère les trame et le planning**
- **effectif :**
- **Diagramme Entité-Relation (ERD) : Comment les objets sont liés entre eux (ex: Un utilisateur possède plusieurs commandes).**

5. Design et Interface (Frontend)

- **Wireframes : Croquis basse fidélité de la disposition des éléments.**
- **Charte graphique : Couleurs (codes HEX), typographies et logo.**
- **Maquettes HD : Liens vers Figma ou Adobe XD si applicable.**

6. Stack Technique

- **Langages & Frameworks : (ex: React Native, Flutter, Node.js).**
- **Base de données : (ex: PostgreSQL, Firebase).**
- **Hébergement : (ex: AWS, Vercel).**

7. Plan de Développement (Roadmap)

- **Phase 1 : Conception et Design.**
- **Phase 2 : Développement du Backend et API.**
- **Phase 3 : Intégration Frontend.**
- **Phase 4 : Tests (QA) et Déploiement.**

Conseil d'expert

Dans ton Google Doc, n'hésite pas à utiliser des tableaux pour la section Dictionnaire des données afin de lister clairement les types (String, Boolean, Integer).

Souhaites-tu que je développe une section spécifique (comme le dictionnaire des données ou un exemple de User Flow) selon l'idée de ton application ?

📋 Design Template : Système de Génération de Trame RH (12 Semaines)

1. Lexique & Domain-Driven Design (DDD)

Pour que l'équipe parle le même langage avant de coder.

Terme	Définition
Besoin (Requirement)	Nombre d'agents requis par type de jour (Lundi-Dimanche) défini par le Manager.
Quotité (Capacity)	Temps de travail contractuel d'un agent (ex: 100%, 80%, 50%).
Trame Maître (Master Pattern)	Séquence théorique de 84 jours (12 semaines) composée de Shifts et de Repos.
Rotation (Offset)	Décalage appliqué à chaque agent sur la Trame Maître pour assurer la couverture quotidienne.
Shift	Vacation de travail de 12h (peut être déclinée en Jour/Nuit).
Cycle de Vie	La période totale de 12 semaines avant que la rotation ne recommence.

Exporter vers Sheets

---

2. Modèles de Données (Mocks JSON)

Structure des données à utiliser pour le développement du Frontend.

A. L'Agent (Resource)

JSON

```
{
  "id": "uuid",
  "nom": "Dupont",
  "quotite": 1.0,
  "temps_travail_hebdo": 35,
  "offset_depart": 0 // Position dans le cycle de 12 semaines
}
```

B. Le Besoin (Demand)

JSON

```
{
  "jour": "Lundi",
  "agents_requis": 5,
  "heures_requises": 60
}
```

C. La Trame (Pattern)

JSON

```
{
  "id_trame": "T1",
  "duree_jours": 84,
  "sequence": ["12h", "Repos", "Repos", "12h", "..."]
}
```

---

3. Use Cases (Cas d'Utilisation)

UC-01 : Définition de la Charge

- **Acteur :** Manager.
- **Action :** Saisir le nombre d'agents nécessaires pour chaque jour de la semaine (Ex: 5 le lundi, 2 le dimanche).
- **Résultat :** Le système calcule le volume d'heures total nécessaire sur 12 semaines.

UC-02 : Génération de la Trame Équitable

- **Acteur :** Système.
- **Action :** Répartir les heures de travail sur 84 jours de manière à ce que chaque agent ait le même nombre de repos et de travail à la fin du cycle.
- **Contrainte :** La somme des agents "actifs" chaque jour doit correspondre au **Besoin (UC-01)**.

UC-03 : Visualisation Individuelle

- **Acteur :** Agent.
- **Action :** Consulter son planning projeté sur les 12 prochaines semaines.
- **Logique :** Application de l'index de rotation (`Date + Offset`) sur la Trame Maître.

---

4. Architecture de l'Interface (Frontend)

Écran 1 : Dashboard Configuration (Input)

- **Composant A :** Grille de saisie du besoin (7 colonnes pour les jours de la semaine).
- **Composant B :** Liste des agents avec curseur pour la quotité (%).

Écran 2 : Visualiseur de Trame (Output)

- **Composant C :** Matrice de 12 semaines montrant la succession des shifts de 12h.
- **Composant D :** Indicateur de santé (Check de couverture : est-ce que le besoin est rempli chaque jour ?).

---

5. Analyse du Fichier Source (`planing 12.ods`)

Ce que nous extrayons du document pour le template :

+1

- **Format des Shifts :** Utilisation stricte de blocs de 12h pour maximiser le repos entre les sessions.
- **Logique de Repos :** Le template doit intégrer les patterns de repos type "2-2-3" ou "Grande/Petite semaine" identifiés dans les structures de roulement classiques.
- **Calcul de conformité :** Utilisation des sommes d'heures présentes dans le fichier pour créer les validateurs du frontend.

  +1

📋 Design Template : Système de Génération de Trame RH (12 Semaines)

1. Lexique & Domain-Driven Design (DDD)

Pour que l'équipe parle le même langage avant de coder.

Terme	Définition
Besoin (Requirement)	Nombre d'agents requis par type de jour (Lundi-Dimanche) défini par le Manager.
Quotité (Capacity)	Temps de travail contractuel d'un agent (ex: 100%, 80%, 50%).
Trame Maître (Master Pattern)	Séquence théorique de 84 jours (12 semaines) composée de Shifts et de Repos.
Rotation (Offset)	Décalage appliqué à chaque agent sur la Trame Maître pour assurer la couverture quotidienne.
Shift	Vacation de travail de 12h (peut être déclinée en Jour/Nuit).
Cycle de Vie	La période totale de 12 semaines avant que la rotation ne recommence.

Exporter vers Sheets

---

2. Modèles de Données (Mocks JSON)

Structure des données à utiliser pour le développement du Frontend.

A. L'Agent (Resource)

JSON

```
{
  "id": "uuid",
  "nom": "Dupont",
  "quotite": 1.0,
  "temps_travail_hebdo": 35,
  "offset_depart": 0 // Position dans le cycle de 12 semaines
}
```

B. Le Besoin (Demand)

JSON

```
{
  "jour": "Lundi",
  "agents_requis": 5,
  "heures_requises": 60
}
```

C. La Trame (Pattern)

JSON

```
{
  "id_trame": "T1",
  "duree_jours": 84,
  "sequence": ["12h", "Repos", "Repos", "12h", "..."]
}
```

---

3. Use Cases (Cas d'Utilisation)

UC-01 : Définition de la Charge

- **Acteur :** Manager.
- **Action :** Saisir le nombre d'agents nécessaires pour chaque jour de la semaine (Ex: 5 le lundi, 2 le dimanche).
- **Résultat :** Le système calcule le volume d'heures total nécessaire sur 12 semaines.

UC-02 : Génération de la Trame Équitable

- **Acteur :** Système.
- **Action :** Répartir les heures de travail sur 84 jours de manière à ce que chaque agent ait le même nombre de repos et de travail à la fin du cycle.
- **Contrainte :** La somme des agents "actifs" chaque jour doit correspondre au **Besoin (UC-01)**.

UC-03 : Visualisation Individuelle

- **Acteur :** Agent.
- **Action :** Consulter son planning projeté sur les 12 prochaines semaines.
- **Logique :** Application de l'index de rotation (`Date + Offset`) sur la Trame Maître.

---

4. Architecture de l'Interface (Frontend)

Écran 1 : Dashboard Configuration (Input)

- **Composant A :** Grille de saisie du besoin (7 colonnes pour les jours de la semaine).
- **Composant B :** Liste des agents avec curseur pour la quotité (%).

Écran 2 : Visualiseur de Trame (Output)

- **Composant C :** Matrice de 12 semaines montrant la succession des shifts de 12h.
- **Composant D :** Indicateur de santé (Check de couverture : est-ce que le besoin est rempli chaque jour ?).

---

5. Analyse du Fichier Source (`planing 12.ods`)

Ce que nous extrayons du document pour le template :

+1

- **Format des Shifts :** Utilisation stricte de blocs de 12h pour maximiser le repos entre les sessions.

- **Logique de Repos :** Le template doit intégrer les patterns de repos type "2-2-3" ou "Grande/Petite semaine" identifiés dans les structures de roulement classiques.
- **Calcul de conformité :** Utilisation des sommes d'heures présentes dans le fichier pour créer les validateurs du frontend.

  +1

🎨 Design Template : Front-End React (Planning Generator)

1. Lexique & DDD (Domain Driven Design)

- **Need (Le Besoin)** : Valeur d'entrée (input) fixant le nombre d'agents nécessaires par jour (ex: Lundi = 5).

  +2
- **Master Pattern (Trame Maître)** : Tableau de 84 jours (12 semaines) représentant le cycle complet avant répétition.

  +2
- **Agent Offset (Décalage)** : L'index de départ (0 à 83) assigné à chaque agent pour qu'ils ne travaillent pas tous en même temps.

  +1
- **Compliance (Conformité)** : Vérification que le cumul d'heures sur 12 semaines correspond au contrat de l'agent (ex: 100% = 35h/semaine en moyenne).

  +1

---

2. Modèle de données : `master_pattern.json`

Ce fichier contient la séquence théorique de 84 jours. Pour tes tests React, ce JSON simule la réponse de ton futur Backend.

JSON

```
{
  "pattern_name": "Roulement 12h Standard",
  "cycle_duration": 84,
  "time_unit": "days",
  "sequence": [
    {"day": 1, "type": "WORK", "duration": 12},
    {"day": 2, "type": "REST", "duration": 0},
    {"day": 3, "type": "REST", "duration": 0},
    {"day": 4, "type": "WORK", "duration": 12},
    {"day": 5, "type": "WORK", "duration": 12},
    {"day": 6, "type": "REST", "duration": 0},
    {"day": 7, "type": "REST", "duration": 0},
    {"day": 8, "type": "REST", "duration": 0},
    {"day": 9, "type": "WORK", "duration": 12},
    {"day": 10, "type": "WORK", "duration": 12},
    {"day": 11, "type": "REST", "duration": 0},
    {"day": 12, "type": "REST", "duration": 0},
    {"day": 13, "type": "WORK", "duration": 12},
    {"day": 14, "type": "WORK", "duration": 12}
    // ... La séquence continue jusqu'au jour 84
  ]
}
```

---

3. Use Case Frontend : Génération de l'affichage

Titre : Projection de la trame sur le calendrier des agents.

- **Données en entrée** : `currentDate`, `agentOffset`, `masterPattern`.
- **Logique React** :
  1. Calculer le jour du cycle : `const cycleDay = (diffDays + agentOffset) % 84`.

2. Récupérer le type de shift dans le JSON pour cet index.

- **Résultat** : Une vue calendrier où chaque agent voit son roulement de 12 semaines se répéter.

  +2

---

4. Architecture des Composants React

Pour ton développement, structure ton application ainsi :

- `<DemandConfig />` : Formulaire permettant de saisir le nombre d'agents par jour (Lundi à Dimanche).

  +2

- `<ResourceList />` : Liste des agents mockés avec leur quotité (100%, 80%).

  +1

- `<PlanningMatrix />` : Le coeur du projet. Une grille (CSS Grid ou Table) qui affiche :
  - En ordonnée (Y) : Les agents.
  - En abscisse (X) : Les jours (sur 12 semaines).
  - Cellule : Affiche "12h" ou "Vide" selon le calcul de l'Offset.

---

5. Logique de "Validation du Besoin"

Dans ton code React, tu devras implémenter cette fonction de calcul pour vérifier ton besoin :

JavaScript

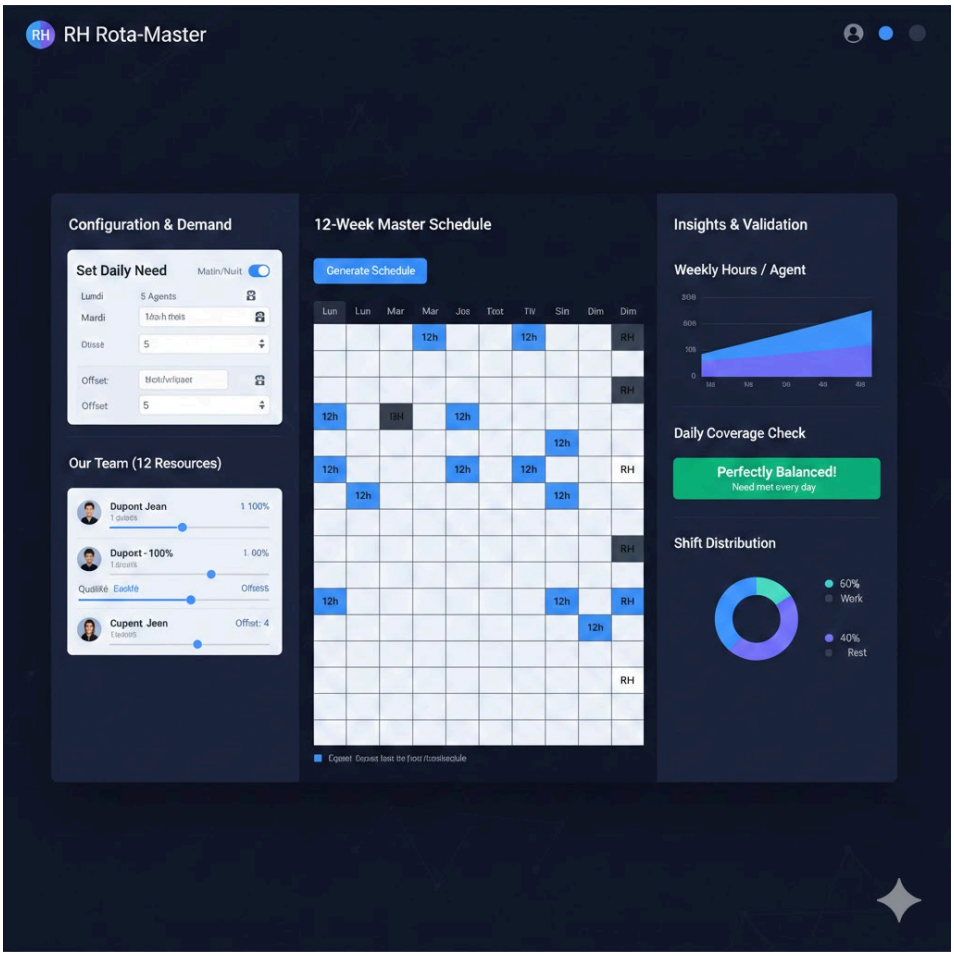
```
// Pseudo-code pour le validateur
const checkCoverage = (day, agents) => {
  const countPresent = agents.filter(agent => isWorkingOn(day, agent.offset)).length;
  const required = demandPerDay[day.name];
  return countPresent >= required; // Retourne true si le besoin est couvert
}
```

Ce que l'on tire du fichier `.ods`

Le fichier original servait de base pour les durées de 12h et la répétition bimensuelle. En passant sur une trame de 12 semaines, ton application React permettra une bien meilleure équité, car elle lissera les week-ends travaillés sur une période beaucoup plus longue que ce que permettait le fichier Excel statique.

+1

Souhaites-tu que je te génère le "Mock" de la liste des agents (avec différents offsets) pour peupler ton interface React immédiatement ?



Voici un bloc structuré, prêt à être copié-collé directement dans ton document Notion. Il fait le pont entre les règles légales dont nous venons de parler et leur traduction technique dans ton application.

Tu peux l'insérer à la suite de ton **Design Template** (par exemple en section 6).

🛡️ 6. Moteur de Conformité Légale (Compliance Engine)

Objectif : Garantir que les trames générées par l'algorithme respectent le Code du Travail et les conventions hospitalières (FPH, CCN 51) avant d'être validées par le manager.

A. Référentiel des Règles Métier (Business Rules)

Ces règles agissent comme des "gardiens" lors de la génération de la Trame Maître.

ID Règle	Règle Légale / Conventionnelle	Traduction Technique (Algorithme)	Criticité
BR-01	**Durée max. quotidienne :** 12h max.	`shift.duration <= 12`	Bloquant 🔴
BR-02	**Repos quotidien :** 11h consécutives min.	`start_shift_B - end_shift_A >= 11h`	Bloquant 🔴
BR-03	**Plafond hebdomadaire :** 48h absolues max / semaine.	`sum(hours) per week <= 48`	Bloquant 🔴
BR-04	**Lissage hebdomadaire :** 44h en moyenne.	`sum(hours) / 12 weeks <= 44`	Bloquant 🔴
BR-05	**Repos dominical :** 1 dimanche sur 2 en repos.	`count(Sunday_OFF) >= (Total_Weeks / 2)`	Avertissement 🟠

Exporter vers Sheets

B. Impact sur l'Architecture (Backend - Django)

- **Création d'un module `validators.py` :** L'algorithme de génération de la trame (la matrice) passe d'abord par ce module.
- **Logique de rejet :** Si une règle `Bloquante` échoue, le backend annule la sauvegarde en base de données et renvoie une erreur HTTP 400 (Bad Request) avec le détail de la règle enfreinte.

C. Impact sur l'Interface (Frontend - React)

- **Feedback Visuel (UI) :** Ajout d'une jauge de "Santé du Planning" sur le composant `<DashboardConfig />`.
- **Gestion des erreurs :** Si le manager demande 10 agents tous les jours mais n'a qu'un pool de 12 agents, le Frontend intercepte l'erreur 400 du Backend et affiche un *Toast* (notification) :

> *"⚠️ Impossible de générer la trame : Le besoin saisi dépasse le plafond légal de 48h/semaine pour vos agents."*

🛰️ 7. Spécifications de l'API (Django REST Framework)

C'est ici que tu définis comment React va demander les données au serveur.

A. Endpoint : Génération de la Trame Maître

- **URL :** `/api/v1/planning/generate-master/`
- **Méthode :** `POST`
- **Payload (Input) :**JSON

```
{
  "daily_requirements": [5, 5, 5, 5, 5, 2, 2], // Lun-Dim
  "total_agents": 12,
  "cycle_weeks": 12
}
```

- **Réponse (Output) :** Un objet contenant la trame de 84 jours validée.

B. Endpoint : Récupération du Planning Global

- **URL :** `/api/v1/planning/full-view/`
- **Méthode :** `GET`
- **Paramètres :** `?start_date=2024-01-01&months=3`
- **Réponse :** Une liste d'agents avec leurs noms et leurs shifts calculés par l'algorithme de rotation.

⚛️ 8. Architecture du Composant React Principal

Pour ton front-end, voici la structure recommandée pour le composant qui va piloter l'affichage.

Composant : `MasterPlanningMatrix.jsx`

Ce composant doit recevoir la trame et les agents, puis faire le calcul d'affichage.

JavaScript

```
import React from 'react';

const MasterPlanningMatrix = ({ agents, masterPattern, startDate }) => {
  // Fonction utilitaire pour calculer le shift d'un agent à une date T
  const getShiftForAgent = (agentOffset, targetDate) => {
    const diffInDays = Math.floor((targetDate - startDate) / (1000 * 60 * 60 * 24));
    const patternIndex = (diffInDays + agentOffset) % 84;
    return masterPattern[patternIndex];
  };

  return (
    <div className="planning-grid">
      {/* En-tête des dates */}
      <header className="grid-header">...</header>

      {/* Lignes par Agent */}
```

```
      {agents.map(agent => (
        <div key={agent.id} className="agent-row">
          <span>{agent.name}</span>
          {/* Cellules des jours */}
          {daysArray.map(day => (
            <div className="shift-cell">
              {getShiftForAgent(agent.offset, day)}
            </div>
          ))}
        </div>
      ))}
    </div>
  );
};
```

🛠️ 9. Analyse Technique : Pourquoi l'approche "Offset" est supérieure à ton ODS ?

Dans ton fichier `planing 12.ods`, le planning est "figé" dans des cellules. Si tu veux décaler une équipe, tu dois tout réécrire.

Avec cette modélisation pour ton app :

1. **Changement d'équipe éclair :** Si l'agent A veut échanger sa rotation avec l'agent B, tu as juste à inverser leurs `rotation_offset` en base de données. Le planning entier se met à jour instantanément.
2. **Simulation de besoin :** Le manager peut changer le "Besoin" (ex: passer de 5 à 6 agents le lundi) et le Backend lui dira immédiatement s'il doit recruter ou si sa trame actuelle peut absorber la charge.
3. **Scalabilité :** Que tu aies 10 ou 1000 agents, la complexité du calcul reste la même (O(1) pour l'accès à un shift donné).

Conclusion du dossier de modélisation

Tu as maintenant :

1. Le **Lexique DDD**.
2. Les **Modèles de données** (JSON & Django).
3. L'**Algorithme** de Master Pattern.
4. Les **Règles Légales** (Infirmiers/12h).
5. Le **Plan React** et les **Endpoints API**.
6.

C'est une précision fondamentale. En ajoutant la notion de **surnuméraire** (overstaffing) et les **spécificités des shifts (7h30/10h30 vs 12h)**, on passe d'un simple calendrier à un véritable outil de gestion hospitalière.

Voici comment intégrer ces deux dimensions dans ton plan Notion.

🏗️ 10. Gestion des Congés et Surnuméraire (Overstaffing)

L'approche "Surnuméraire" consiste à concevoir une trame où le nombre d'agents affectés est délibérément supérieur au besoin strict. Cela permet d'absorber les congés sans modifier la structure du roulement.

Le Concept Technique :

- **Trame Théorique :** Conçue pour N+1 ou N+2 agents.
- **Pool de Remplacement :** Les agents en surnuméraire sur la trame occupent les "cases vides" laissées par ceux en congés.
- **Priorité d'Affichage :** Le système affiche d'abord la trame théorique, puis applique une couche "Congés" qui libère une place, immédiatement comblée par l'agent en position de surnombre ce jour-là.

⚙️ 11. Configuration des Shifts (Options du Créateur)

Le manager peut choisir entre deux modèles de journée lors de la création de l'agenda. Voici la modélisation des deux options :

Option A : Modèle Hybride (7h30 + 10h30)

- **Shift Matin :** 7h30
- **Shift Après-midi :** 7h30
- **Shift Nuit (ou Long) :** 10h30
- **Transmission :** 1h30 de superposition (inclus dans le calcul de présence).
- **Logique de calcul :** Le backend doit vérifier que la somme des heures sur 24h respecte le chevauchement pour garantir la continuité des soins.

Option B : Modèle Linéaire (12h)

- **Shift :** 12h pile.
- **Transmission :** Incluse dans les 12h (pas de superposition supplémentaire).
- **Logique de calcul :** Plus simple mathématiquement, mais plus contraignante sur le repos de 11h.

📝 Bloc à ajouter sur ton Notion (Template de Design)

Copie-colle ce bloc pour finaliser ta partie "Configuration Avancée".

🛠️ Configuration des Cycles et Shifts

Le créateur d'agenda définit ici les règles structurelles du roulement.

1. Choix du Type de Vacation

Type	Composition	Transmission	Total Cycle (24h)
Standard	2 × 7h30 + 1 × 10h30	1h30 (Superposition)	25h30 cumulées
Continu	2 × 12h	Incluse	24h00 cumulées

Exporter vers Sheets

2. Stratégie de Couverture (Congés)

- **Mode Surnuméraire :** Le système autorise l'affectation de X agents supplémentaires sur la Trame Maître.
- **Logique d'Absorption :** * Si `Agent_A` pose un congé → Statut passe à `CONGÉ`.
  - Le système cherche l'agent en position `REPOS_SURNUMÉRAIRE` sur la même trame.
  - Le système propose automatiquement le remplacement sans casser la rotation des autres agents.

3. Algorithme de Calcul des Heures (Pseudo-code)

Python

```
if config.shift_type == "HYBRIDE":
    daily_total = (7.5 * 2) + 10.5 # 25.5h
    transmission_overlap = 1.5
else:
    daily_total = 12 * 2 # 24h
    transmission_overlap = 0
```

🧪 Pourquoi c'est crucial pour ton Backend Django ?

En Django, tu devras ajouter un champ `shift_category` dans ton modèle `PlanningConfig`.

- Si `HYBRIDE`, ton validateur de 48h/semaine devra compter **25h30** de travail produit sur une journée complète (répartie sur 3 agents) pour vérifier le budget de l'unité.

- Le "Surnuméraire" sera simplement un flag sur l'objet `Agent` (ex: `is_backup = True`).

Veux-tu que je te fasse le schéma visuel du chevauchement de 1h30 pour le modèle 7h30+10h30 pour l'expliquer graphiquement dans ton Notion ?